SESSION 04 / PRINTABLE GUIDE

Lesson 1: When a notebook earns its place

Session 04 | Notebooks with Warehouse Gold
Output: A requirement-based choice of notebook, Warehouse SQL, or a hybrid.

For the primary hands-on workload, use the realistic notebook lab: 50 devices over fourteen days and 5,000 nested orders. Lessons 02-07 retain small fixtures to explain individual rules, not to demonstrate analytical scale. First-time users should complete Notebook 101 and its repair exercise before that workload. Use the importable starter/solution pairs, not a blank notebook.

Do not frame this as Python replacing SQL

A notebook is an interactive code document executed by a runtime. In these examples it runs PySpark over files and Delta tables in a Lakehouse. A Warehouse is a relational analytical store with a SQL development and serving model. They are complementary components, not equivalent storage engines.

Use a notebook when its programming model, libraries, or processing pattern materially improves the solution. Keep a SQL-first approach when the problem is already relational, the required operations are supported, and the team can operate it more simply in SQL.

SQL can parse JSON, perform window calculations, and aggregate events. A CSV, a large row count, or the existence of an array is not by itself proof that Spark is required. The decision combines source complexity, transformation needs, scale, operational skills, dependencies, and support.

Useful customer scenarios

Use case	Why a notebook can help	Warehouse's role	What does not justify a notebook
Nested API/export payloads	Traverse repeated arrays and structures, enforce schemas, retain raw payloads, and share Python parsing functions	Serve flat order lines, dimensions, and measures through approved SQL/BI contracts	A small stable JSON feed that an existing supported low-code/SQL path handles well
Device telemetry and log files	Prepare many files, normalize timestamps, apply reusable event checks, and aggregate accepted events with Spark	Serve device/day facts and operational summaries	Claiming notebooks are automatically faster for every dataset

Use case	Why a notebook can help	Warehouse's role	What does not justify a notebook
Specialized standardization	Reuse an approved address, text, or domain-specific Python library with versioned tests	Store normalized attributes, review decisions, and approved matching results	Basic trim/case conversion already supported in SQL/ Power Query
Entity resolution	Generate candidate matches, compute specialized similarity scores, and retain explanations	Publish reviewed crosswalks and canonical dimensions	Automatically merging people/vendors because names look similar
Model scoring or feature engineering	Use a verified model/ library to calculate features or scores in a controlled environment	Serve versioned scores/ features alongside business dimensions	Calling an external model without data approval or inventing an available AI feature
Geospatial/file-specific processing	Apply approved coordinate/file libraries where supported SQL functions do not meet the need	Serve derived regions, distances, or business attributes	Treating notebook availability as proof that required native libraries are installed

The two worked examples use nested orders and telemetry. They need no third-party packages. Specialized libraries and model scoring are discussion use cases requiring separate dependency, licensing, data, and tenant review.

Example 1: Nested order exports

An application supplies one JSON document per order. Customer attributes are nested; each order contains an array of lines. Quantities and prices arrive as strings. Some line keys are repeated, some quantities cannot be interpreted, and an order may have no lines.

The notebook preserves each raw document, expands the line array, applies explicit types and key rules, and separates accepted lines from quarantine. Warehouse Gold exposes the approved flat line contract.

Why use a notebook here? The lesson focuses on a reusable file/structure-processing path that can grow into multiple payload versions and parsing functions. It does not claim that a Warehouse cannot parse JSON.

Gold grain: One accepted `(order_id, line_number)` in the current approved teaching snapshot.

Business boundary: Accepted lines are not automatically a complete financial ledger. Quarantined lines need review; totals must be labeled as accepted-line totals.

Example 2: Device events

Devices export readings with event identifiers, UTC timestamps, and temperature strings. Repeated event keys, malformed timestamps, nonnumeric readings, and values outside the agreed range need explicit treatment before daily summaries are calculated.

The notebook keeps the raw events, normalizes identifiers/time, quarantines every occurrence of an ambiguous duplicate key, and aggregates the accepted events by device and UTC day. The micro fixture produces two daily rows; the main lab produces 700 across fourteen days.

Why use a notebook here? This demonstrates reusable event preparation over file-oriented data, with a path to more files and specialized feature calculations. It is not a streaming implementation and is not justified by nine fixture rows.

Gold grain: One `(device_id, reading_date)` for the current approved teaching snapshot.

Business boundary: A day summary over accepted observed readings does not prove complete device coverage or absence of missing events.

Why keep Gold in a Warehouse?

The operating team may prefer SQL for serving contracts, curated tables, relational modeling, change review, and downstream BI. That is a valid reason to combine notebook preparation with Warehouse publication.

It also introduces a real transfer boundary. Lakehouse Delta commits, Warehouse copy activity, SQL publication, model freshness, and consumer access are separate operations. Sharing OneLake does not make the Warehouse Gold table automatically appear or give all these operations one transaction.

Avoid moving the same business rule between Spark, SQL, and DAX without naming its owner. In these examples Spark owns parsing and acceptance rules; Warehouse SQL owns validation of the transfer contract and publication; the semantic model owns reporting measures over the published result.

When Warehouse SQL is the better starting point

Stay SQL-first for supported relational joins, dimensional transformations, simple normalization, and reporting-oriented logic when the data is already available in the Warehouse and the team can support it.

Session 03's two-file reconciliation is the counterexample: rewriting it in Python would not add a demonstrated requirement. Notebook execution adds Spark startup, runtime management, dependency/version concerns, another permission path, and a skill/support burden.

Notebook concepts to teach before the demos

Teach these through the warm-up and repair exercise, including three short practice cells, equivalent Spark SQL/PySpark summaries, and optional safe persistence/restart verification.

1. Python variables are not durable storage. A successful cell does not mean a table was persisted.
2. A Spark DataFrame is a distributed computation plan; actions and writes execute work. Avoid collecting large datasets to the driver.
3. pandas and PySpark are not interchangeable. pandas generally processes data in one process; PySpark distributes supported work.
4. Run cells from a clean session in order. Hidden state from earlier experiments is not a repeatable workflow.
5. Use a named batch, exact input path, explicit schema, and approved identity. Do not hide secrets in cells or output.
6. A default Lakehouse supplies context for relative paths and unqualified table names. It is not the Warehouse.
7. Overwriting one Delta table does not atomically overwrite every output table or publish Warehouse Gold.

Decision exercise

Ask participants to select a tool and explain the rejected alternative:

- Two relational extracts need a supported full outer join and fixed comparison rules.
- Hundreds of nested payload files need a reusable parser and accepted/quarantine outputs.
- A specialized matching library produces candidate vendor pairs that require human approval.
- Daily device facts can be computed either from Warehouse tables with SQL or from raw event files with Spark.

A strong answer cites the actual source and transformation requirements, the team's skills, verified tenant features, and a condition that would cause the decision to change.

Next: Notebook 101 for beginners, then the main notebook lab. Use the nested JSON micro example first when learners need to predict each row before working with the larger batch.

Source: 01_notebook_use_cases.md. Companion-material links open the repository originals.